Informe del Sprint 3

Automatización de pruebas de la API de cadetería

Proyecto: Especialización en Herramientas de Testing (TAP)
Equipo: QualityTech Solutions — Basauri Augusto, Garnica German, Medina Jorge, Vallejo Leandro
Herramienta: Postman + Newman
Sistema bajo prueba: API REST del sistema de cadetería (Java 21 / Spring Boot 3.3.5, MySQL)
Fecha: 2026-10-27

Resumen ejecutivo

En el Sprint 3 se tomó la colección manual del Sprint 2 —15 casos sobre 5 endpoints de la API REST de cadetería— y se la convirtió en una suite automatizada y repetible con Postman + Newman, con reportes HTML/JSON y ejecución continua en GitHub Actions.

Resultado: la suite pasó de 35 a 90 aserciones por ejecución, sumó chequeos globales (tiempo de respuesta, Content-Type y formato estándar de error), quedó repetible gracias a un paso de limpieza final y se ejecuta sin intervención manual. Validación medida el 2026-10-27 contra el mock del contrato: 19 requests, 90 aserciones, 0 fallos, 1685 ms en total y 3 ms de respuesta promedio.

1. Documento de optimización

1.1 Punto de partida (colección del Sprint 2)

- 18 requests (15 casos + P1, P2, P3), 18 scripts de test.
- 35 aserciones en total; cada caso de error repetía a mano las mismas verificaciones.
- Credenciales y URL dentro de la colección; ids inexistentes escritos a mano en los casos 08 y 14.
- El caso 10 cambiaba el estado del cadete a DESCONECTADO y no lo restauraba: una segunda ejecución partía de un estado distinto.
- Sin límite de tiempo de respuesta ni verificación del formato de error en todos los requests.
- Ejecución manual desde Postman.

1.2 Cambios realizados

#	Cambio	Justificación técnica
1	Chequeos a nivel de colección: tiempo de respuesta, Content-Type JSON y formato estándar de error (status/error/codigo/message)	Se escriben una vez y se aplican a los 19 requests; antes no existían. Detecta regresiones de contrato sin tocar cada caso
2	Aserciones nuevas: expiresAt (01), nombre y apellido (04), mismo cadete (10), número de pedido (13)	Verifican el contenido de la respuesta, no solo el código HTTP
3	Chequeo de que la respuesta no expone la contraseña (04 y 07)	Control de seguridad barato sobre datos del cadete
4	Paso Z1 de limpieza: restaura el estado del cadete a LIBRE	La suite queda repetible; el pipeline no depende del estado anterior
5	Variables idInexistente y maxMs	Sin valores fijos en los casos; el umbral de tiempo se ajusta por entorno
6	Credenciales y URL en archivos de entorno (local y ci)	La misma colección corre en la máquina de cada integrante y en el pipeline
7	Newman + reporters HTML/JSON, scripts npm y workflow de GitHub Actions	Ejecución y reportes sin intervención manual
8	Resumen automático (scripts/resumen.js)	Resultados visibles en la pestaña del workflow sin abrir el reporte

1.3 Medición antes / después

Métrica	Antes	Después
Requests	18	19 (se suma Z1)
Casos de prueba	15	15
Aserciones por ejecución	35	90
Chequeos de tiempo de	0	19

respuesta		
Verificaciones de formato de error	0 (cada caso verificaba codigo a mano)	10 (una por cada respuesta de error, definidas una sola vez)
Valores fijos en los casos	2 ids + credenciales	0
Estado del sistema tras ejecutar	modificado (estado del cadete)	restaurado
Pasos manuales para ejecutar y reportar	abrir Postman, importar, correr, armar el reporte a mano	0 (npm test o push al repositorio)

1.4 Ejecución medida (Newman, 2026-10-27)

Métrica	Antes (Postman manual)	Después (Newman)
Tiempo para ejecutar y obtener el reporte	~10–15 min (importar, correr los 18 requests y armar el reporte a mano)	~1,7 s de ejecución; los reportes HTML y JSON se generan solos
Tiempo de respuesta promedio de la API	no se registraba	3 ms (mín 1 ms, máx 30 ms)
Casos aprobados / fallidos	15/15 aprobados (revisión manual, sin reporte)	15/15 aprobados · 90/90 aserciones · 0 fallos

Nota: los valores de la columna «después» son reales y medidos contra el mock del contrato (19 requests, 90 aserciones, 0 fallos, 1685 ms en total). El mock valida que la suite y el pipeline funcionan, pero no describe el rendimiento de la API real: esos tiempos deben re-medirse contra el backend (job api-real) y promediar 3 corridas. Los valores «antes» son estimaciones del trabajo manual del Sprint 2.

2. Automatización completa funcional

La colección Postman «Instituto - Suite automatizada API cadetería (Sprint 3)» contiene 19 requests: 15 casos (5 endpoints × 3 casos), 3 pasos de preparación (P1–P3) y 1 de limpieza (Z1). Corre completa y en orden con Newman y no necesita intervención manual.

Cobertura de la suite

Endpoint	Casos
POST /api/auth/login/cadete	01 OK · 02 401 CREDENCIALES_INVALIDAS · 03 400 VALIDACION
GET /api/cadetes/me	04 OK · 05 401 NO_AUTENTICADO · 06 401 token vencido
GET /api/admin/cadetes/{id}	07 OK · 08 404 NO_ENCONTRADO · 09 403 SIN_PERMISO
PATCH /api/cadetes/me/estado	10 OK · 11 400 VALIDACION · 12 401 NO_AUTENTICADO
GET /api/pedidos/me/{id}	13 OK · 14 404 NO_ENCONTRADO · 15 403 SIN_PERMISO

P1 y P2 inician sesión (admin y cadete) y guardan tokens; P3 obtiene el id de un pedido del historial; Z1 restaura el estado del cadete a LIBRE. Chequeos globales de colección: tiempo de respuesta < maxMs (1500 ms por defecto), Content-Type JSON y, en las respuestas de error, formato estándar status / error / codigo / message.

Scripts npm

Script	Qué hace
npm run mock	Levanta mock/server.js (servidor simulado del contrato)
npm test	Newman contra la API real (entorno local), reportes HTML/JSON
npm run test:ci	Newman contra el mock (entorno ci), reportes HTML/JSON
npm run resumen	Arma el resumen en Markdown a partir de reportes/reporte.json

Ejecución

```
git clone <URL-del-repositorio>
cd <carpeta>
npm install

# Contra la API real (levantarla antes, con la carga de demostración):
npm test

# Contra el mock (sin backend):
npm run mock      # en una terminal
npm run test:ci   # en otra

# Resumen en consola:
npm run resumen
```

3. Configuración CI/CD

El workflow .github/workflows/tests.yml se dispara en cada push a main, en cada pull request y a pedido (workflow_dispatch). Tiene dos jobs.

Job 1 — suite-contra-mock (corre siempre)

- actions/checkout@v4 y actions/setup-node@v4 (Node 20).
- npm install.
- Levanta el mock (nohup node mock/server.js) y espera el health check /actuator/health.
- Ejecuta la colección con Newman (npm run test:ci).
- Genera el resumen (npm run resumen) aunque haya fallos (if: always()).
- Sube reportes/ y mock.log como artifact «reportes-mock».

Job 2 — api-real (corre contra el backend real)

Se activa al definir la variable del repositorio BACKEND_REPO (y el secreto BACKEND_TOKEN si el repositorio es privado). Levanta MySQL 8.0 como service, compila el backend con Maven (actions/setup-java@v4, Java 21), lo arranca apuntando a MySQL y ejecuta la misma colección, subiendo reportes/ y api.log como artifact «reportes-api-real».

4. Reportería

- Reporters de Newman: cli, htmlextra y json.
- Salidas: reportes/reporte.html (visual) y reportes/reporte.json (datos).
- scripts/resumen.js lee el JSON, calcula requests, aserciones, duración y tiempo promedio, y lo imprime como tabla Markdown; en GitHub Actions también lo agrega a $GITHUB_STEP_SUMMARY.
- En el pipeline, los reportes quedan como artifacts de cada ejecución (pestaña Actions).

Evidencia de la ejecución automática (2026-10-27)

Métrica	Valor
Requests ejecutados	19
Aserciones	90 OK / 0 fallidas
Duración total	1685 ms
Tiempo de respuesta promedio	3 ms
Resultado	Sin fallos

5. Bitácora actualizada

Fecha	Integrante	Tarea	Decisión / problema	Resolución
2026-10-27	Basauri Augusto	Consolidar la colección y agregar chequeos globales	Los 15 casos de error repetían a mano las mismas verificaciones y no había control de tiempo, Content-Type ni formato completo del error	Chequeos a nivel de colección (tiempo, Content-Type, formato de error) + aserciones de contenido nuevas
2026-10-27	Garnica German	Workflow de GitHub Actions	Ejecutar la suite sin depender del backend en cada push, y contra la API real solo cuando esté disponible	tests.yml con dos jobs: suite-contra-mock (siempre) y api-real (se activa con BACKEND_REPO, levanta MySQL + backend)
2026-10-27	Medina Jorge	Mediciones antes/después y documento de optimización	Mostrar el impacto con números (aserciones, pasos manuales, repetibilidad)	docs/ optimizacion.md con tabla antes/después (35 → 90 aserciones, 0 → 19 chequeos de tiempo, 0 pasos manuales)
2026-10-27	Vallejo Leandro	README, reportes y prueba desde cero	Que cualquier integrante (o el evaluador) pueda clonar y correr todo sin contexto previo	README con ejecución desde cero, scripts/resumen.js y validación de instalación limpia contra el mock

Decisiones técnicas

- Credenciales y URL en archivos de entorno (local y ci) y no en la colección: pm.variables.get resuelve entorno + colección.
- Tokens e ids como variables de colección; no definirlos en el entorno, porque el valor del entorno tiene prioridad y pisaría lo que guardan los scripts.
- Chequeos transversales (tiempo, formato, seguridad) como reglas globales de colección, no repetidos por caso.
- Ejecución contra mock en el pipeline para validar la suite sin el backend; la API real corre en el job api-real.

- Umbral de tiempo (maxMs) e ids inexistentes (idInexistente) parametrizados, sin valores fijos en los casos.
- Paso Z1 de limpieza para que la suite sea repetible.

Problemas resueltos

Problema	Causa	Solución
Una segunda ejecución fallaba o partía de un estado distinto	El caso 10 cambiaba el estado del cadete a DESCONECTADO y no lo restauraba	Paso Z1 que restaura el estado a LIBRE al final
Un token guardado por un script aparecía vacío o pisado	Prioridad de variables en Postman: el valor del entorno pisa al de la colección	Tokens e ids solo como variables de colección, sin definirlos en el entorno
Los casos 08 y 14 tenían ids inexistentes escritos a mano	Ids fijos dentro del request	Variable idInexistente reutilizada en ambos casos
Ejecutar y reportar a mano era lento y propenso a error	Postman manual + armado del reporte a mano	Newman con reporters cli/htmlextra/json; npm test; scripts/resumen.js
No había forma de validar la suite sin el backend	Dependencia del backend real	Mock del contrato (mock/server.js) que corre en el job suite-contra-mock

6. Repositorio Git

Todo el trabajo está versionado y el README documenta el uso. Estructura del repositorio:

Ruta	Qué es
postman/coleccion-cadeteria.postman_collection.json	Colección: 15 casos + P1–P3 + Z1
postman/entorno-local.postman_environment.json	URL de la API y usuarios de prueba (local)
postman/entorno-ci.postman_environment.json	Lo mismo para el pipeline
.github/workflows/tests.yml	Pipeline de integración continua
mock/server.js	Servidor simulado del contrato de la API
scripts/resumen.js	Arma el resumen de resultados a partir del reporte JSON
docs/	Documento de optimización, bitácora y análisis
reportes/	Se genera solo (no se versiona)

Requisitos

- Node.js 20 o superior.
- La API levantada en http://localhost:8080 con la carga de demostración (admin, cadete 30111222 y pedidos de ejemplo), o el mock incluido.

.gitignore

```
node_modules/
reportes/
*.log
```

7. Análisis

¿Qué mejoraron?

- Más y mejores aserciones: de 35 a 90 por ejecución; se pasó de validar solo el código HTTP y el codigo de error a verificar contenido (expiresAt, nombre/apellido, número de pedido) y que no se expone la contraseña (04 y 07).
- Chequeos que antes no existían y ahora se aplican a todos los requests: tiempo < maxMs, Content-Type JSON y formato estándar de error (19 verificaciones de tiempo y 10 de formato definidas una sola vez).
- Ejecución sin pasos manuales: npm test o un push dispara todo y genera reportes HTML/JSON más un resumen.
- Suite repetible: Z1 restaura el estado del cadete y el sistema queda igual antes y después.
- Parametrización y portabilidad: ids y credenciales fuera de los casos; entornos separados local y ci.
- Integración continua: el workflow corre en cada push/PR y un fallo deja el pipeline en rojo.

¿Qué costó más?

- Que la suite dependa de la carga de demostración y del orden de ejecución (P1–P3 antes de los casos; el caso 06 asume una sola sesión por cadete).
- Levantar el backend con MySQL dentro del pipeline (job api-real): compilar con Maven, arrancar MySQL y esperar a que el backend responda.
- La prioridad de variables en Postman: una variable de entorno vacía pisa el valor que guarda un script; costó depuración entender por qué un token aparecía vacío.

¿Qué aprendieron de la automatización?

- Probar a mano no es lo mismo que mantener infraestructura de pruebas: la colección pasó a ser código que se versiona, se revisa y se mantiene.
- Qué automatizar y qué no: los chequeos transversales rinden más como reglas globales que repetidas por caso; el orden y los datos de prueba son parte del diseño.
- Un pipeline en rojo evita que un error llegue a producción: cada push/PR re-ejecuta los 15 casos y bloquea la integración ante regresiones.
- El valor de un mock para desacoplar la validación de la suite de la disponibilidad del backend.

Anexo A. Resumen de ejecución (consola)

```
## Resultado de la suite de pruebas (Newman)


| Métrica | Valor |
|---|---|
| Requests ejecutados | 19 |
| Aserciones | 90 OK / 0 fallidas |
| Duración total | 1685 ms |
| Tiempo de respuesta promedio | 3 ms |

Sin fallos.
```